

1. What "training a model" means

An LLM is a very large set of numbers called **weights**. Qwen2.5-1.5B has about 1.5 billion of them. Before training, these numbers are random, and the model outputs random tokens.

Training means adjusting these numbers, repeated millions of times:

```
Give the model some text
→ model predicts the next token
→ compare prediction with the actual next token → compute error (loss)
→ adjust every weight slightly to reduce that error
→ repeat with the next piece of text
```

Pre-training and fine-tuning both use this exact process. **They differ in the starting weights, the data, the scale and the goal, not in the method.** If students remember one thing from this topic, it should be this.

Weights are adjustable numbers inside the model that decide how input numbers are turned into output numbers. Training is the process of finding the values that make the model's outputs correct.

A neuron takes some input numbers, multiplies each by a weight, adds them up, and adds one more number called the **bias**:

$$\text{output} = (w1 \times x1) + (w2 \times x2) + b$$

Say we want to predict how likely a customer is to redeem their mutual fund investment. Higher output means more likely.

- $x1$ = how much the market fell this month (%): **3**
- $x2$ = years the customer has been invested: **10**
- $w1 = 0.8$, $w2 = -0.3$, $b = 1$

$$\text{output} = (0.8 \times 3) + (-0.3 \times 10) + 1 = 2.4 - 3 + 1 = 0.4$$

Here the weights have a readable meaning:

- **Sign gives direction.** w_1 is positive, so a bigger market fall increases the output. w_2 is negative, so more years invested decreases it.
- **Size gives strength.** A weight near 0 means that input barely affects the output. A large weight (positive or negative) means it has a big effect.

This is where "weights = importance" comes from, and for a tiny model with meaningful inputs, it is roughly correct.

One detail: real neurons also pass this sum through an **activation function** (for example, turning negative values into 0). This allows the network to learn patterns that are not straight lines. The weights work the same way.

****Meaning is spread across many weights.**** There is no single weight for "NAV" or for "grammar". The model's knowledge of what NAV is comes from millions of weights working together, and each weight also contributes to many unrelated things. Changing one weight out of billions usually changes nothing noticeable.

Pre-training

Starting point: random weights.

Data: trillions of tokens of raw text (web pages, books, Wikipedia, code, research papers). No human labels anything. Qwen2.5 was pre-trained on about 18 trillion tokens, Llama 3 on about 15 trillion.

How the labels come from the text itself: take one sentence, "The fund's NAV rose today". It automatically produces these training examples:

Input the model sees	Correct next token
The	fund
The fund	's
The fund's	NAV
The fund's NAV	rose
The fund's NAV rose	today

Every sentence on the internet produces its own questions and answers this way. This is called **self-supervised learning**, and it is why pre-training can use so much data: no one has to label it.

What the model learns: to predict the next token well across trillions of examples, it has to learn grammar, facts, reasoning patterns, code syntax, and several languages. None of

these are taught directly. They are needed to predict well.

Cost: thousands of GPUs running for weeks to months. The largest Llama 3 model (405B) used about 16,000 H100 GPUs. Costs run into tens of millions of dollars. Only large organisations do this: Meta, Alibaba (Qwen), Google, Mistral, DeepSeek, Anthropic, OpenAI.

Output: a base model. It can continue text, but it **does not follow instructions**. Ask a base model "What is NAV in mutual funds?" and it may continue with "What is AUM? What is SIP? What is an expense ratio?", because in its training data a question like that is often part of a list of FAQ questions. It is continuing text, not answering you.

The step in between (done by the model company)

Before we get to your fine-tuning, there is a stage in the middle that freshers usually don't know about. After pre-training, the company fine-tunes the base model itself:

```
Random weights
  → PRE-TRAINING (trillions of tokens, raw text)          → Base model
e.g. Qwen2.5-1.5B
  → SFT (instruction fine-tuning: question → good answer)
  → Preference tuning (RLHF / DPO: better vs worse answer) → Instruct
model   e.g. Qwen2.5-1.5B-Instruct
  → YOUR FINE-TUNING (thousands of your examples)       → Your model
```

Both versions are on Hugging Face as separate downloads: Qwen2.5-1.5B (base) and Qwen2.5-1.5B-Instruct. **The instruct model is itself already a fine-tuned model.** When students fine-tune, they are almost always adding one more fine-tuning round on top of the company's.

The stages above are together called **post-training**: everything done after pre-training to make the model useful.

4. Fine-tuning

Starting point: the weights of an already pre-trained model (usually the instruct version).

Data: hundreds to a few thousand **curated input** → **output examples** for one specific task. For example:

```
{"messages": [
  {"role": "user", "content": "Classify this complaint: My SIP amount was debited twice this month."},
  {"role": "assistant", "content": "{\"category\": \"Duplicate Debit\",
```

```
\\"priority\\": \\"High\\""}"}  
  ]}
```

Process: the same next-token prediction and weight updates. The model predicts each token of the answer, the error is measured, and the weights are adjusted. Two practical differences from pre-training: the learning rate is lower (so existing knowledge is not overwritten), and the loss is usually calculated only on the answer tokens, not the question.

Cost: with LoRA (covered later), a 1.5B model on 1,000 examples trains in roughly 15–30 minutes on a free Colab T4 GPU.

What fine-tuning is good at: changing **behaviour**. It can make the model always output a fixed JSON format, write in a specific tone, classify into your own categories, follow your company's rules for answering, or use your domain's terminology correctly.

****What fine-tuning is poor at: adding new facts**** reliably. A few thousand examples showing a fact is far less than the trillions of tokens the model's knowledge came from. The model may learn the fact partly, mix it up with similar facts, or state it wrongly with confidence. For facts, use RAG.

	Pre-training	Fine-tuning
Starting weights	Random	Already pre-trained
Data	Raw text, no labels	Curated input → output pairs
Data size	Trillions of tokens	Hundreds to thousands of examples
Who creates the labels	The text itself (next token)	People or a stronger LLM
Hardware	Thousands of GPUs	1 GPU (often a free one)
Time	Weeks to months	Minutes to hours
Cost	Tens of millions of dollars	Near zero to a few thousand rupees
Who does it	Large AI labs	Any engineer
Goal	General language ability and knowledge	One specific task or behaviour
Training method	Next-token prediction	Next-token prediction (same)
Output	Base model	Specialised model

One more type: continued pre-training

Between the two, there is **continued pre-training** (also called domain-adaptive pre-training). You take a pre-trained model and continue pre-training it on **raw, unlabelled text from one domain**, for example millions of pages of SEBI circulars, annual reports and fund factsheets. The method is the same as pre-training (next token on raw text), but the scale is much smaller.

Use it when the model understands your domain's language poorly, for example a regional language or very specialised terminology. It needs far more data and compute than fine-tuning, so it is rarely the first choice. Mention it to students so they know it exists, and keep the focus on fine-tuning.

Base vs Instruct models

(Collab : https://colab.research.google.com/drive/1foyC28r45lAY0voFTaO4cQ4-jNzsmLA_?authuser=1#scrollTo=iXGfkPeBCiOS)

Base Model :

A **base model** has only learned to continue text. An **instruct model** is that same base model, trained further so it answers what you ask.

It is trained on one task: read text, predict the next word. It is trained on trillions of words from websites, books and code.

So whatever you type, it treats as the start of a document and writes what would probably come next in that document.

Pre-training (next-token prediction)	→ BASE model
+ instruction fine-tuning (+ preference tuning)	→ INSTRUCT / CHAT model
+ extra training on multi-step problems	→ REASONING model

1. Base model

How it is made: only pre-trained. It learned to predict the next token on a very large amount of text.

What it does: it can complete text and has the general abilities it picked up during pre-training. But it may continue the wording of your prompt instead of treating it as a request.

Use it for: research and further fine-tuning. It is usually awkward to use directly as a chatbot.

TinyLlama example: `TinyLlama-1.1B-intermediate-step-1431k-3T` . Give it "What is NAV in mutual funds?" and it continues the text instead of answering.

2. Instruct (or Chat) model

How it is made: it starts from the base model and is trained further on instructions paired with good responses. Many teams then add a preference-tuning step (such as DPO) after that.

What it does: it is more likely to answer the question directly, follow the format you ask for, and use the chat template of its model family.

Use it for: normal chat and prompt-following tasks.

TinyLlama example: `TinyLlama-1.1B-Chat-v1.0` . It is the base model after SFT on UltraChat and DPO on UltraFeedback. Give it the same question and it answers, then stops.

3. Reasoning model

How it is made: this category is less standardised. It usually means a model trained specifically for problems that need several intermediate steps. Its training may use reinforcement learning on tasks with checkable answers (for example maths with a known result), copying the outputs of a stronger reasoning model (distillation), or both.

What it does: before giving the final answer, it usually writes out many more tokens of step-by-step working.

Trade-off: this extra working helps with maths, programming and other multi-step tasks, but it makes responses slower and more expensive, and it wastes effort on simple questions.

Use it for: difficult tasks that benefit from working through steps.

TinyLlama has no reasoning version. Small examples from other families: `DeepSeek-R1-Distill-Qwen-1.5B` , or Qwen3 models, which can switch thinking on or off.

Note:

Just like pre-training, there is post training - SFT + DPO

Post-training is everything done to a model after pre-training. For TinyLlama Chat it had two steps: **SFT** first, then **DPO**.

BASE model (continues text)

→ SFT: learn to answer, from examples of good answers

→ DPO: learn which answers are better, from pairs of good vs bad

answers

→ CHAT model

Step 1: SFT (Supervised Fine-Tuning)

Goal: teach the base model to answer questions and stop, instead of continuing the text.

The data: a large set of conversations, each with a user message and a good assistant reply. For TinyLlama Chat this was UltraChat: conversations generated by ChatGPT.

User: What is NAV in mutual funds?

Assistant: NAV (Net Asset Value) is the price of one unit of a mutual fund.

Step A: each conversation is written in the chat template (TinyLlama's format):

```
<|user|>
```

```
What is NAV in mutual funds?</s>
```

```
<|assistant|>
```

```
NAV (Net Asset Value) is the price of one unit of a mutual fund.</s>
```

`</s>` means **end of sequence**, not end of sentence.

Sequence: a complete piece of text, which can have many sentences. In a chat, it is one full message.

Step B: training is the same next-token prediction as pre-training. The model reads the text and predicts each next token. The difference is that **the error is only counted on the assistant's part:**

Model sees: `<|user|> What is NAV in mutual funds? </s> <|assistant|>`

Must predict: "NAV" → error counted

Then sees ... "NAV", must predict "(Net" → error counted

...

Then sees the full answer, must predict "</s>" → error counted

or another example

Step	Model has seen so far	Correct next token	Error counted?			
1	`<	user	> What is NAV? <			
2	... `<	assistant	> NAV`			
3	... `<	assistant	> NAV is`			
4	... `<	assistant	> NAV is the`			
5	... NAV is the price	of	✅ Yes			
6	... NAV is the price of	one	✅ Yes			
7	... NAV is the price of one	unit.	✅ Yes			
8	... NAV is the price of one unit.	</s>	✅ Yes			

The question part is given to the model, but its predictions there are not scored. We want it to learn to write answers, not questions.

What the model learns from thousands of examples like this:

- After `<|assistant|>`, write an answer to what the user asked.
- When the answer is complete, output `</s>`, and generation stops.

This fixes both problems in your base model output: repeating the question, and never stopping.

What SFT cannot teach: each training example shows only **one** good answer. The model is never shown a bad answer, so it never learns what makes one answer better than another. After SFT, the model answers, but it can still give answers that are unhelpful, too long, or careless.

Step 2: DPO (Direct Preference Optimization)

Goal: teach the model to prefer better answers over worse ones.

The data: each example has one question and **two answers**, one marked **chosen** (better) and one **rejected** (worse):

Question: Should I invest all my savings in one small-cap fund?

Chosen: Putting all savings in one small-cap fund is high risk, because


```
small-cap
    funds can fall sharply. Spreading money across different fund
types
    reduces this risk.

Rejected: Yes, small-cap funds give the highest returns. Go for it.
```

For TinyLlama Chat this was UltraFeedback: about 64,000 questions, each answered by several models, with the answers ranked by GPT-4. A top-ranked answer is used as "chosen", a lower-ranked one as "rejected".

What training does with each pair:

- 1. The model calculates how likely it would be to write the chosen answer, and how likely it would be to write the rejected answer. (It gives a probability to each token of an answer, and these combine into a probability for the whole answer.)
- 2. The weights are adjusted so the **chosen answer becomes more likely** and the **rejected answer becomes less likely**.
- 3. A frozen copy of the model from after SFT is kept for comparison. The training measures changes relative to this copy, so the model does not drift too far from what SFT taught it (it should still answer and stop).

What the model learns: among the many answers it could write, prefer the kind people (or a judge model) rate higher: more accurate, more helpful, more careful.

Why "Direct": the older method, **RLHF**, first trains a separate model to score answers (a reward model), then uses reinforcement learning to train the chat model against those scores. DPO skips the separate scoring model and learns directly from the chosen/rejected pairs. It is simpler and cheaper, which is why it is widely used.

	SFT	DPO
Data per example	Question + one good answer	Question + chosen answer + rejected answer
What the model learns	How to answer and when to stop	Which answers are better
How	Predict the answer's tokens, error counted on the answer only	Make chosen answer more likely, rejected less likely
Order	First	After SFT
TinyLlama Chat data	UltraChat (ChatGPT conversations)	UltraFeedback (64k questions, GPT-4 ranked)

Special token terms you will come across

1. Core tokens (almost every model has these)

Term	Full name	One-liner
BOS	Beginning of sequence	Marks the start of the input text.
EOS	End of sequence	Marks where the text ends; when the model generates it, generation stops.
PAD	Padding	Filler added to shorter sentences in a batch so all rows are the same length; ignored via the attention mask.
UNK	Unknown	Used for text the tokenizer cannot represent (rare in byte-level tokenizers, which can represent anything).

2. Chat tokens (instruct / chat models)

Term	One-liner
Role markers	Mark who is speaking: system, user or assistant.
Start of turn	Marks where one message in the conversation begins.
End of turn (EOT)	Marks where one message ends; for the assistant's message, it also stops generation.
System token / system role	Marks the instructions that set the model's behaviour for the whole conversation.

3. Special-purpose tokens (only some models)

Term	One-liner
Tool-call tokens	Wrap the JSON the model writes when it wants to call a function.
Tool-response tokens	Wrap the function's result when it is sent back to the model.
Thinking tokens	Wrap the step-by-step working of a reasoning model, before the final answer.
FIM tokens (fill-in-the-middle)	Used by code models to fill a gap between code before and code after the cursor.
Vision / image tokens	Mark where an image's data sits inside the text, in models that accept images.

4. Older tokens (BERT-style models, not text generation)

Term	One-liner
CLS	Classification: placed at the start; its output is used to classify the whole text.
SEP	Separator: placed between two pieces of text, for example a question and a passage.
MASK	Hides a word during training so the model learns to guess it

Same job, different names in each model family

Job	TinyLlama / Llama 2	Llama 3	Qwen2.5	Gemma	BERT
BOS	<s>	`<	begin_of_text	>`	none
EOS	</s>	`<	end_of_text	>`	`<
PAD	none set	none set	`<	endoftext	>`
UNK	<unk>	none	none	<unk>	[UNK]
Start of turn	`<	user	> , <	assistant	>` (from chat template)
End of turn	</s>	`<	eot_id	>`	`<
Tool call	none	through the chat template	<tool_call> </tool_call>	none	n/a
Thinking	none	none	none (Qwen3 has <think> </think>)	none	n/a

The names differ, but the jobs are the same. This is why you always use each model's own tokenizer and `apply_chat_template()` , instead of writing these tokens by hand.

For TinyLlama specifically: `<|user|>` and `<|assistant|>` are not special tokens in the vocabulary. They are ordinary text written by the chat template, split into normal tokens. Only `<s>` , `</s>` and `<unk>` are true special tokens.

Refer this collab now :

<https://colab.research.google.com/drive/1J8P8hVNv28SmsLcLuY9BNn1ig8cPPPe-?authuser=1#scrollTo=GNzw6YxoKebY>

Attention masking

First: what "attention" means

When the model processes a token, it does not look at that token alone. It **looks at the other tokens in the sentence** and combines information from them. This is called **attention**.

For sentence 1, when the model processes `?`, it looks at:

```
<s> What is N AV ?
```

It uses these tokens to work out what `?` means in this sentence: it ends a question about NAV. Every token does this, for every layer of the model.

The problem padding creates

After padding, sentence 1 has 4 filler tokens:

```
<s> What is N AV ? <unk> <unk> <unk> <unk>
```

Without any extra instruction, the model would treat the fillers as part of the sentence and look at them too. The information it combines would include 4 meaningless tokens. So:

- The numbers it calculates for sentence 1 would be different from processing "What is NAV?" alone.
- Its output would change just because a longer sentence happened to be in the same batch.

What the attention mask does

The mask tells the model, for each position, **whether other tokens may look at it**:

Tokens:	<s>	What	is	N	AV	?	<unk>	<unk>	<unk>	<unk>
Mask:	1	1	1	1	1	1	0	0	0	0

- **1** = real token, may be looked at
- **0** = filler, must not be looked at

How it works inside: before combining information, the model gives every token a score for how much to use it. For positions where the mask is 0, the score is set so that the token gets **zero weight**. The fillers then contribute nothing.

Result: sentence 1 is processed exactly as if the fillers were not there. Its output is the same whether it is sent alone or in a batch.

In one sentence

Padding makes every row the same length so sentences can be processed together, and the attention mask makes the model ignore the padding, so the extra tokens don't change the result.

Another mask :

There is a second, separate mask inside every text-generation model, the **causal mask**. It stops each token from looking at tokens **after** it:

```
When processing "is", the model may look at:  <s>  What  is
                                         but not at:      N  AV  ?
```

This is needed because the model is trained to predict the next token. If it could see the tokens ahead, it would just copy the answer instead of predicting it. The model applies this mask automatically, so you never create it yourself. The attention mask from the tokenizer is the one you pass in, and it only deals with padding.

Base versus Chat

	Base model	Chat model
Input	Plain question	Question in the chat template
First token it writes	Repeats the question	Starts an answer
During generation	Question → answer → question → answer...	One answer
How it stops	Hits the 80-token limit	Generates <code></s></code>
New tokens	80 of 80	Fewer than 80
Same model size, same architecture, same tokenizer. The only difference is the extra training (SFT + DPO), which changed the weights so that after <code>< assistant ></code> the model answers and then stops.		